Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with 'A' Grade (CGPA : 3.15)

Department of Artificial Intelligence and Machine Learning
B.Tech. Sem: V Subject: Full Stack Development Using NextJs (DJS23AMD302L)
EXPERIMENT NO. 7

Name : Devyani · S · Kamat SAP : 60017240102

AIM:

To implement secure social authentication using Better Auth with Google and GitHub OAuth providers on a Next.js platform.

SOFTWARE REQUIRED:

- **Operating System:** Linux / macOS / Windows 10/11
- **Database Engine:** PostgreSQL (running via Docker Compose or local instance)
- **Node.js runtime environment:** (v18.17.0 or higher)
- **Package Manager:** npm or pnpm
- **Code Editor:** Visual Studio Code
- **Developer Accounts:** Google Cloud Console & GitHub Developer Settings

THEORY:

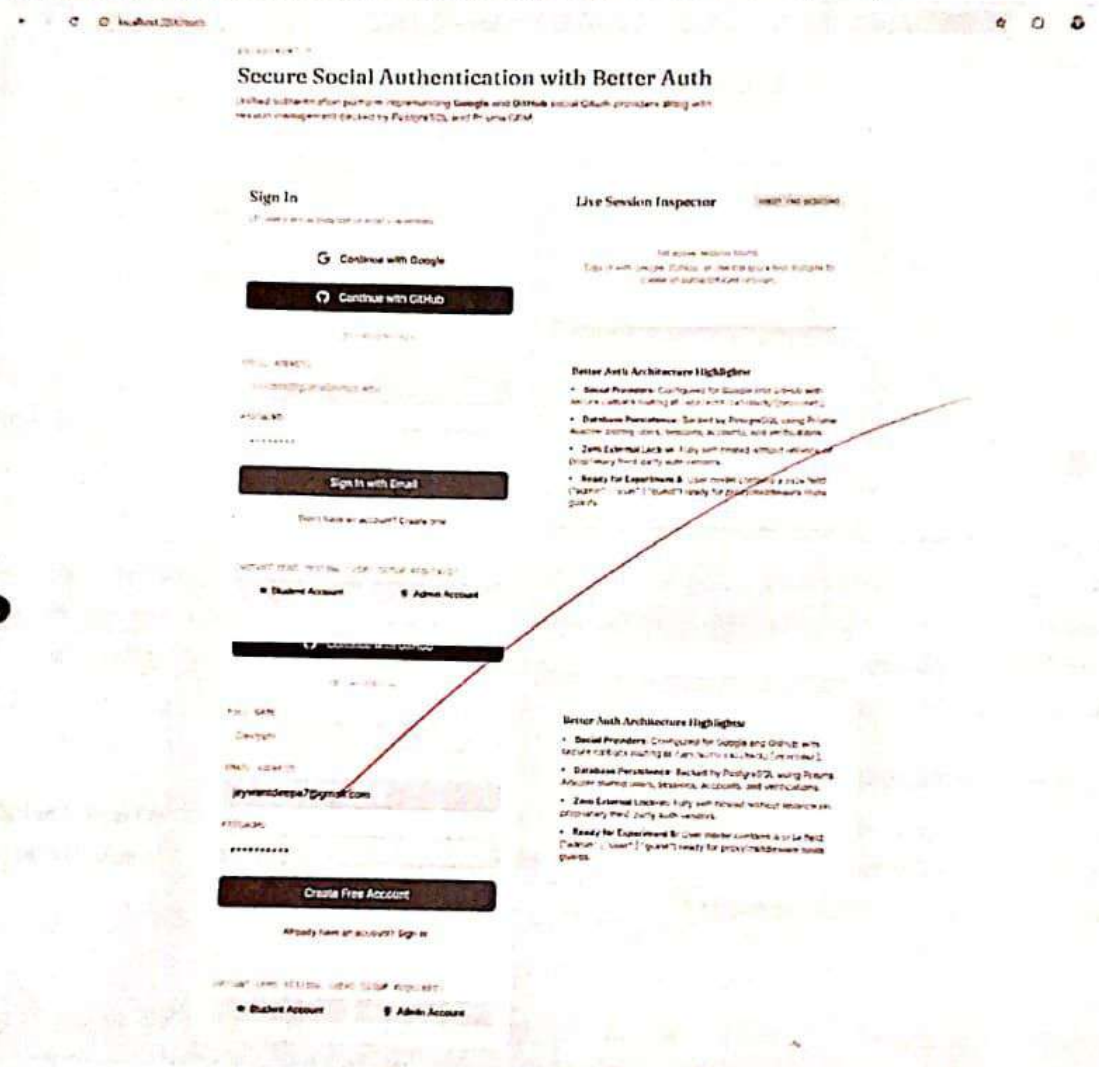
- **Modern Authentication with Better Auth:** Better Auth is a TypeScript-first, framework-agnostic authentication framework tailored for Next.js App Router. It abstracts session handling, OAuth 2.0/OIDC handshakes, CSRF mitigation, and database persistence into a modular server configuration with typed client libraries.
- **OAuth 2.0 Authorization Code Flow:** Social login follows the OAuth 2.0 Authorization Code Grant:
  1. The client initiates login, redirecting the user's browser to the provider (Google/GitHub) with client_id, requested scope, and a cryptographic state token to prevent CSRF attacks.
  2. The user authenticates and approves basic profile permissions (openid, profile, email).
  3. The provider redirects back to the configured application callback URL (/api/auth/callback/[provider]) with an authorization code.
  4. Better Auth exchanges the authorization code on the server for an access_token and identity payload, creates or links database records (user, account, session), and issues a secure, HTTP-only session cookie.
- **Database Persistence Strategy:** Better Auth maintains normalized tables to handle identity lifecycles:
  - user: Stores canonical user identities (id, name, email, avatar).
  - session: Stores active sessions with expiry timestamps and associated token hashes.
  - account: Maps third-party OAuth provider identifiers (google, github) and provider account IDs to internal user IDs.
  - verification: Stores transient tokens for email verification and one-time password handshakes.

Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA 3.18)

Department of Artificial Intelligence and Machine Learning

B.Tech. Sem: V Subject: Full Stack Development Using NextJs (DJS23AMD302L)

Open browser Developer Tools (Application $\rightarrow$ Storage $\rightarrow$ Cookies) to verify the encrypted, HTTP-only cookie (better-auth.session_token). Refreshing the browser preserves the active session state without triggering re-authentication prompts.



Observation:

Verified OAuth 2.0 verification code handshakes, secure server-side session management using PostgreSQL & client-side session state handling.



CONCLUSION:

Secure social authentication using Better Auth with Google and GitHub OAuth providers was successfully implemented on Next.js. The experiment verified OAuth 2.0 authorization code handshakes, secure server-side session management using PostgreSQL, and client-side session state handling using typed hooks.

POST LAB QUESTION:

Q: Why are OAuth 2.0 client secrets stored strictly in server-side environment variables rather than exposed to the client-side Next.js bundle (NEXT_PUBLIC_*), **and what attack vector does this prevent?**

A:

OAuth client secrets are used by the backend application to authenticate itself against the identity provider when exchanging a temporary authorization code for user access tokens.

If client secrets are exposed to the client bundle (e.g., via NEXT_PUBLIC_*), malicious actors can extract the secret and impersonate the application. An attacker could forge token exchange requests, intercept sensitive profile tokens, or register unauthorized redirect URIs, completely compromising user authentication and enabling account takeover attacks. Keeping credentials strictly server-side ensures the token-exchange step remains confidential.

Conclusion:
Secure social authentication using Better Auth with Google & GitHub OAuth providers was successfully implemented on Next.js